

Refatoração e Melhorias do Código

<https://gemini.google.com/app/c1b87f8e9389b23b>

```
User prompt: package br.edu.infnet.herick_bispo_blog_API; import
br.edu.infnet.herick_bispo_blog_API.domain.model.Artigo; import
br.edu.infnet.herick_bispo_blog_API.domain.model.Autor; import
br.edu.infnet.herick_bispo_blog_API.domain.model.Comentario; import
br.edu.infnet.herick_bispo_blog_API.domain.model.Leitor; import
org.springframework.boot.CommandLineRunner; import org.springframework.stereotype.Component;
import java.util.ArrayList; import java.util.List; @Component public class ProjetoRunner implements
CommandLineRunner { @Override public void run(String... args) throws Exception { // - Instanciação
Autor e Leitor
```

```
-----
//Instanciação de Autor e Leitor
        Autor autor1 = new Autor(1L,"herick",
"herick@gmail.com");    Leitor leitor1 = new Leitor(2L, "hater", "hater@gmail.com");
leitor1.assinarNewsLetter(true);    System.out.println("\n");    System.out.println(autor1);
System.out.println(leitor1); // - Artigos
```

```
-----
//Publicar Artigo    Artigo artigo1 = new Artigo(3L,
"Estudar depois da aula", "Estudar depois da aula é importante para fixar o conteúdo.");
autor1.publicarArtigo(artigo1);    //Listar Todos os Artigos - Pssivelmente será refatorado com streams
        List<Artigo> repositorioDeArtigos = new ArrayList<>();    repositorioDeArtigos.add(artigo1);
//Excluir Artigos    var excluirArtigo = false;    if (excluirArtigo == true) {
autor1.excluirArtigo(artigo1, excluirArtigo);    repositorioDeArtigos.remove(artigo1);    }
System.out.println("\n");    System.out.println(artigo1); // - Comentários
```

```
-----
//Listar e Moderar Comentários
        leitor1.comentar(4L, artigo1,"Seu texto está cheio de erros
de português (¬_¬)");    leitor1.avaliar(artigo1, 1.0);    System.out.println("\nLista de pendentes antes da
moderação: ");    artigo1.getComentariosPendentes().forEach(System.out::println);
System.out.println("\nLista de aprovados antes da moderação: ");
artigo1.getComentarios().forEach(System.out::println); // - Moderar Comentários
```

```
-----
//Moderar Comentários
        Comentario comentario1 =
artigo1.getComentariosPendentes().get(0);    autor1.moderarComentario(artigo1, comentario1, true);
System.out.println("\nLista de pendentes depois da moderação: ");
artigo1.getComentariosPendentes().forEach(System.out::println);    System.out.println("\nLista de
aprovados depois da moderação: ");    artigo1.getComentarios().forEach(System.out::println);    //Artigo
impresso com os comentários moderados    System.out.println("\n");    System.out.println(artigo1); // -
Exibir Artigos
```

```
-----
//Possivelmente será refatorado com streams para usar
apenaa uma lista    repositorioDeArtigos.forEach(System.out::println);    }} package
br.edu.infnet.herick_bispo_blog_API.domain.service; import java.util.HashMap; import java.util.Map; public
abstract class BaseService<T extends Identificavel> { private final Map<Long, T> dados = new
```

```

HashMap<Long, T>()); } package br.edu.infnet.herick_bispo_blog_API.domain.model; import
java.time.LocalDateTime; import java.util.ArrayList; import java.util.List; public class Artigo implements
Identificavel{ private Long id; private String titulo; private String conteudo; private LocalDateTime
dataPublicacao; private boolean publicado; private int visualizacoes = 0; private double avaliacao =
0.0; private int quantidadeAvaliacoesArtigo = 0; private double somaAvaliacoesArtigo = 0.0; private
Autor autor; // Será refatorado com streams para usar apenas uma lista private List<Comentario>
comentarios = new ArrayList<Comentario>(); private List<Comentario> comentariosPendentes = new
ArrayList<Comentario>(); public Artigo(){ } public Artigo(Long id, String titulo, String conteudo) {
this.id = id; this.titulo = titulo; this.conteudo = conteudo; }//
----- public void receberComentario(Comentario comentario) { // Será refatorado com streams para
usar apenas uma lista if (comentario.isAprovado()){ comentarios.add(comentario); } else {
comentariosPendentes.add(comentario); } } public void receberAvaliacao(double nota) {
this.quantidadeAvaliacoesArtigo++; this.somaAvaliacoesArtigo += nota; this.avaliacao =
this.somaAvaliacoesArtigo / this.quantidadeAvaliacoesArtigo; autor.avaliarReputacaoAutor(avaliacao);
} public void incrementarVisualizacao(){ this.visualizacoes++; }//
----- @Override public String toString() { return String.format("Artigo {titulo= '%s', conteudo= '%s',
dataPublicacao= %s, autor= %s, comentarios= %d}", titulo, conteudo,
dataPublicacao, autor.getNome(), comentarios.size() ); } public Long getId() {
return id; } public void setId(Long id) { this.id = id; } public String getTitulo() { return titulo; }
public void setTitulo(String titulo) { this.titulo = titulo; } public String getConteudo() { return
conteudo; } public void setConteudo(String conteudo) { this.conteudo = conteudo; } public
LocalDateTime getDataPublicacao() { return dataPublicacao; } public void
setDataPublicacao(LocalDateTime dataPublicacao) { this.dataPublicacao = dataPublicacao; } public
boolean isPublicado() { return publicado; } public void setPublicado(boolean publicado) {
this.publicado = publicado; } public int getVisualizacoes() { return visualizacoes; } public void
setVisualizacoes(int visualizacoes) { this.visualizacoes = visualizacoes; } public Autor getAutor() {
return autor; } public void setAutor(Autor autor) { this.autor = autor; } public List<Comentario>
getComentarios() { return comentarios; } public void setComentarios(List<Comentario>
comentarios) { this.comentarios = comentarios; } public List<Comentario>
getComentariosPendentes() { return comentariosPendentes; } public void
setComentariosPendentes(List<Comentario> comentariosPendentes) { this.comentariosPendentes =
comentariosPendentes; } } package br.edu.infnet.herick_bispo_blog_API.domain.model; import
java.time.LocalDateTime; public class Comentario implements Identificavel{ private Long id; private
String texto; private LocalDateTime dataHora; private boolean aprovado; private Usuario
autorComentario; public Comentario(){ } public Comentario(Long id, String texto) { this.id = id;
this.texto = texto; } @Override public String toString() { return String.format("Comentário {texto=
'%s', autorComentario= %s, dataHora= %s}", texto, autorComentario.getNome(),
dataHora ); } public Long getId() { return id; } public void setId(Long id) { this.id = id; }
public String getTexto() { return texto; } public void setTexto(String texto) { this.texto = texto; }
public LocalDateTime getDataHora() { return dataHora; } public void setDataHora(LocalDateTime
dataHora) { this.dataHora = dataHora; } public boolean isAprovado() { return aprovado; }
public void setAprovado(boolean aprovado) { this.aprovado = aprovado; } public Usuario
getAutorComentario() { return autorComentario; } public void setAutorComentario(Usuario
autorComentario) { this.autorComentario = autorComentario; } } package
br.edu.infnet.herick_bispo_blog_API.domain.model; import java.time.LocalDateTime; public abstract class
Usuario implements Identificavel{ private Long id; private String nome; private String email; private
LocalDateTime dataCadastro; public Usuario(){ } public Usuario(Long id, String nome, String email,
LocalDateTime dataCadastro) { this.id = id; this.nome = nome; this.email = email;
this.dataCadastro = dataCadastro; } @Override public String toString() { return String.format("id=
%d, nome= %s, email= %s, dataCadastro= %s", id, nome, email,

```

```

dataCadastro    ); } public Long getId() {    return id; } public void setId(Long id) {    this.id = id;
} public String getNome() {    return nome; } public void setNome(String nome) {    this.nome =
nome; } public String getEmail() {    return email; } public void setEmail(String email) {    this.email
= email; } public LocalDateTime getDataCadastro() {    return dataCadastro; } public void
setDataCadastro(LocalDateTime dataCadastro) {    this.dataCadastro = dataCadastro; }} package
br.edu.infnet.herick_bispo_blog_API.domain.model; import java.time.LocalDateTime; import
java.util.ArrayList; import java.util.List; public class Autor extends Usuario {    private double reputacao = 5.0;
    private int quantidadeAvaliacoesEmArtigos = 0;    private double somaAvaliacoesEmArtigos = 0.0;
    private List<Artigo> artigos = new ArrayList<>();    public Autor(){}    public Autor(Long id,String nome, String
email) {        super(id, nome, email, LocalDateTime.now());    } //

```

```

----- public void publicarArtigo(Artigo artigo){    if(artigo == null){        throw new
IllegalArgumentException("O artigo não pode ser nulo");    }
artigo.setDataPublicacao(LocalDateTime.now());    artigos.add(artigo);    artigo.setAutor(this);
artigo.setPublicado(true); } public void excluirArtigo(Artigo artigo, boolean excluir){    if (excluir) {
    artigos.remove(artigo);    } } public void avaliarReputacaoAutor(double avaliacao){
quantidadeAvaliacoesEmArtigos ++;    this.somaAvaliacoesEmArtigos += avaliacao;    this.reputacao =
this.somaAvaliacoesEmArtigos / this.quantidadeAvaliacoesEmArtigos; } public void
moderarComentario(Artigo artigo, Comentario comentario, boolean aprovado) {    // O autor só pode
moderar os próprios comentários    if (!this.equals(artigo.getAutor())) {        throw new
IllegalArgumentException("O autor só pode moderar comentários dos próprios artigos.");    }    if
(aprovado){        this.aprovar(artigo, comentario);    } else {        this.rejeitar(artigo, comentario);    }
} // Será refatorado com streams para usar apenas uma lista    private void aprovar(Artigo artigo,
Comentario comentario) {        comentario.setAprovado(true);
artigo.getComentariosPendentes().remove(comentario);    artigo.getComentarios().add(comentario); }
// Será refatorado com streams para usar apenas uma lista    private void rejeitar(Artigo artigo,
Comentario comentario) {        comentario.setAprovado(false);
artigo.getComentariosPendentes().remove(comentario);    } //

```

```

----- @Override public String toString() {    return String.format("Autor {%s, reputacao= '%s', artigos=
'%d'",        super.toString(),        reputacao,        artigos.size()    ); } public double
getReputacao() {    return reputacao; } public void setReputacao(double reputacao) {
this.reputacao = reputacao; } public List<Artigo> getArtigos() {    return artigos; } public void
setArtigos(List<Artigo> artigos) {    this.artigos = artigos; }} package
br.edu.infnet.herick_bispo_blog_API.domain.model; import java.time.LocalDateTime; public class Leitor
extends Usuario {    private boolean inscritoNewsletter;    public Leitor(){}    public Leitor(Long id, String
nome, String email) {        super(id, nome, email, LocalDateTime.now());    } //

```

```

----- public void comentar(Long id, Artigo artigo, String texto) {    Comentario novoComentario =
new Comentario(id, texto);    novoComentario.setAutorComentario(this);
novoComentario.setDataHora(LocalDateTime.now());    artigo.receberComentario(novoComentario); }
public void avaliar(Artigo artigo, double nota) {    //A nota deverá ser entre 1 e 5    if(nota <= 1 || nota
>= 5){        throw new IllegalArgumentException("A nota deve ser entre 1 e 5");    }
artigo.receberAvaliacao(nota); } public void assinarNewsLetter(Boolean inscrever){    if(inscrever){
    this.inscritoNewsletter = true;    } else {        this.inscritoNewsletter = false;    } } //

```

```

----- @Override public String toString() {    return String.format("Leitor {%s, inscritoNewsletter=
'%s'",        super.toString(),        inscritoNewsletter ? "sim": "não"    ); } public boolean
isInscritoNewsletter() {    return inscritoNewsletter; } public void setInscritoNewsletter(boolean
inscritoNewsletter) {    this.inscritoNewsletter = inscritoNewsletter; }} package
br.edu.infnet.herick_bispo_blog_API.domain.model; import java.time.LocalDateTime; public class Leitor
extends Usuario {    private boolean inscritoNewsletter;    public Leitor(){}    public Leitor(Long id, String

```

```
nome, String email) {    super(id, nome, email, LocalDateTime.now());    } //
```

```
-----    public void comentar(Long id, Artigo artigo, String texto) {        Comentario novoComentario =  
new Comentario(id, texto);        novoComentario.setAutorComentario(this);  
novoComentario.setDataHora(LocalDateTime.now());        artigo.receberComentario(novoComentario);    }  
    public void avaliar(Artigo artigo, double nota) {        //A nota deverá ser entre 1 e 5        if(nota <= 1 || nota  
>= 5){            throw new IllegalArgumentException("A nota deve ser entre 1 e 5");        }  
    artigo.receberAvaliacao(nota);    }    public void assinarNewsLetter(Boolean inscrever){        if(inscrever){  
        this.inscritoNewsletter = true;    }else {        this.inscritoNewsletter = false;    }    } //
```

```
-----    @Override    public String toString() {        return String.format("Leitor {%s, inscritoNewsletter=  
'%s'",            super.toString(),            inscritoNewsletter ? "sim": "não"        );    }    public boolean  
isInscritoNewsletter() {        return inscritoNewsletter;    }    public void setInscritoNewsletter(boolean  
inscritoNewsletter) {        this.inscritoNewsletter = inscritoNewsletter;    } } package  
br.edu.infnet.herick_bispo_blog_API.domain.model; public interface Identificavel {    Long getId(); } ##  
Sobre o Projeto Este projeto foi desenvolvido com base no desafio **[Personal Blogging Platform  
API](https://roadmap.sh/projects/personal-blog-api)**, proposto pela plataforma de estudos  
**roadmap.sh**. O objetivo do desafio é construir uma API RESTful de backend para alimentar um blog  
pessoal, implementando as operações fundamentais de CRUD (Criar, Ler, Atualizar e Excluir) para o  
gerenciamento de artigos. #### Requisitos Implementados - Retorno de lista de artigos (com filtros  
opcionais). - Busca de um artigo específico via ID. - Criação de novos artigos. - Atualização de artigos  
existentes via ID. - Exclusão de artigos via ID. #### Arquitetura ![Arquitetura da API do  
Blog](https://github.com/user-attachments/assets/17478486-dda0-486d-a759-ba43a3bcc108) Quero  
refatorar o código modificando os seguintes pontos: - Eliminar a lista comentariosPendentes e deixar  
apenas a lista de comentarios. farei a filtragem pelo atributo aprovado(da classe Comentario) - melhorar as  
lógicas de validação de avaliação de artigo e de reputação do autor. - melhorar demais problemas que  
você encontre de lógica, relação, etc Atue como um tutor técnico focado em conceitos de programação  
para um projeto em Java Spring Boot. Siga rigorosamente as diretrizes abaixo: Contexto do Projeto e  
Raciocínio: Construção Progressiva: O projeto faz parte de uma pós-graduação em Engenharia de  
Software e é construído aula a aula. Assuma que algumas implementações atuais podem estar fora das  
melhores práticas definitivas porque o conhecimento ainda está em construção. Não critique ou corrija  
arquiteturas antecipadamente, a menos que eu peça. Raciocínio Passo a Passo: Estou construindo uma  
linha de raciocínio lógico. Responda estritamente ao passo ou dúvida atual. Não se antecipe entregando a  
solução final ou deduzindo os próximos passos, pois desvios quebram o meu progresso. Diretrizes de  
Resposta: Sem introduções: Comece a resposta diretamente com a explicação, sem saudações ou frases  
de transição. Sem blocos de código: Explique a lógica, a arquitetura ou o conceito utilizando apenas texto  
descritivo. Direto e Sucinto: Use o menor número de palavras possível para garantir uma compreensão  
completa. Foco Estrito: Responda única e exclusivamente ao que foi perguntado. Não forneça contextos  
adicionais, curiosidades ou expansões de assunto não solicitadas. Formatação: Utilize negrito para  
termos-chave e listas para detalhar processos, garantindo a escaneabilidade da leitura.
```

Response: Refatoração da Lista de Comentários

- **Unificação:** Remova a lista de comentários pendentes da classe do artigo, mantendo apenas a lista principal.
- **Inserção:** Ao receber um comentário, adicione-o diretamente a esta lista única. Certifique-se de que o estado inicial do objeto comentário seja de **não aprovado**.
- **Filtragem:** Substitua os métodos de obtenção de listas (aprovados e pendentes) por lógicas que iterem sobre a lista principal e retornem apenas os elementos que correspondam ao estado booleano desejado.
- **Moderação:** O método de aprovação ou rejeição do autor precisará apenas alterar o estado booleano do comentário específico, sem necessidade de removê-lo ou movê-lo entre listas.

Melhoria na Lógica de Avaliação de Artigo

- **Condição de Fronteira:** A validação atual da nota possui um erro lógico clássico. Ela rejeita os valores exatos dos limites. A condição deve bloquear exclusivamente valores que sejam **estritamente menores** que o limite inferior ou **estritamente maiores** que o limite superior.

Melhoria na Lógica de Reputação do Autor

- **Erro Matemático:** Atualmente, a cada novo voto em um artigo, a nova **média do artigo** é tratada como um voto individual e somada ao total do autor. Isso distorce completamente o cálculo ao longo do tempo.
- **Solução Estrutural:** A reputação do autor deve refletir a média das notas de seus artigos. Quando um artigo atualizar a própria média, acione a atualização no autor. O autor, por sua vez, deve iterar sobre a sua **lista de artigos publicados**, somar a média atual de cada um e dividir pela quantidade total de artigos.

Correções Lógicas e Relacionais Adicionais

- **Responsabilidade por Identificadores (IDs):** O método de criar comentários exige a passagem de um ID manual. Em sistemas reais, a geração de IDs é responsabilidade da camada de persistência (banco de dados) ou do serviço. O leitor não deve conhecer ou informar o ID ao criar um comentário.
- **Comparação de Objetos:** O método de moderação verifica se o autor do artigo é o mesmo autor que está tentando moderar utilizando comparação direta de instâncias. Para que isso funcione de forma segura em Java, a classe pai dos usuários deve implementar regras claras de igualdade baseadas no **identificador único (ID)**, evitando falsos negativos ao comparar instâncias em memória diferentes que representam o mesmo registro.

```
User prompt: package br.edu.infnet.herick_bispo_blog_API; import
br.edu.infnet.herick_bispo_blog_API.domain.model.Artigo; import
br.edu.infnet.herick_bispo_blog_API.domain.model.Autor; import
br.edu.infnet.herick_bispo_blog_API.domain.model.Comentario; import
br.edu.infnet.herick_bispo_blog_API.domain.model.Leitor; import
org.springframework.boot.CommandLineRunner; import org.springframework.stereotype.Component;
import java.util.ArrayList; import java.util.List; @Component public class ProjetoRunner implements
CommandLineRunner { @Override public void run(String... args) throws Exception { // - Instanciação Autor e
Leitor
```

```
----- Autor autor1 = new Autor(1L, "herick", "herick@gmail.com");
Leitor leitor1 = new Leitor(2L, "hater", "hater@gmail.com"); leitor1.assinarNewsLetter(true);
System.out.println("\n"); System.out.println(author1); System.out.println(leitor1); // - Artigos
```

```
----- //Publicar Artigo Artigo artigo1 = new Artigo(3L, "Estudar
depois da aula", "Estudar depois da aula é importante para fixar o conteúdo.");
autor1.publicarArtigo(artigo1); //Listar Todos os Artigos - Pssivelmente será refatorado com streams
List<Artigo> repositorioDeArtigos = new ArrayList<>(); repositorioDeArtigos.add(artigo1); //Excluir Artigos
var excluirArtigo = false; if (excluirArtigo == true) { autor1.excluirArtigo(artigo1, excluirArtigo);
repositorioDeArtigos.remove(artigo1); } System.out.println("\n"); System.out.println(artigo1); // -
Comentários
```

```
----- leitor1.comentar(4L, artigo1,"Seu texto está cheio de erros de
português (¬_¬)" ); leitor1.avaliar(artigo1, 1.0); System.out.println("\nLista de pendentes antes da moderação:
"); artigo1.filtrarComentariosAprovados().forEach(System.out::println); System.out.println("\nLista de
aprovados antes da moderação: "); artigo1.getComentarios().forEach(System.out::println); // - Moderar
Comentários
```

```
----- Comentario comentario1 =
artigo1.filtrarComentariosPendentes().get(0); autor1.moderarComentario(artigo1, comentario1, true);
System.out.println("\nLista de pendentes depois da moderação: ");
artigo1.filtrarComentariosAprovados().forEach(System.out::println); System.out.println("\nLista de
aprovados depois da moderação: "); artigo1.getComentarios().forEach(System.out::println); //Artigo
impresso com os comentários moderados System.out.println("\n"); System.out.println(artigo1); // - Exibir
Artigos
```

```
----- // Possivelmente será refatorado com streams para usar
apenaa uma lista repositorioDeArtigos.forEach(System.out::println); } } package
br.edu.infnet.herick_bispo_blog_API.domain.service; import
br.edu.infnet.herick_bispo_blog_API.domain.model.Identificavel; import java.util.HashMap; import
java.util.Map; public abstract class BaseService<T extends Identificavel> { private final Map<Long, T> dados
= new HashMap<Long, T>(); } package br.edu.infnet.herick_bispo_blog_API.domain.model; import
java.time.LocalDateTime; import java.util.ArrayList; import java.util.List; public class Artigo implements
Identificavel{ private Long id; private String titulo; private String conteudo; private LocalDateTime
dataPublicacao; private boolean publicado; private int visualizacoes = 0; private double avaliacao = 0.0;
private int quantidadeAvaliacoesArtigo = 0; private double somaAvaliacoesArtigo = 0.0; private Autor
autor; private List<Comentario> comentarios = new ArrayList<Comentario>(); public Artigo(){ public
Artigo(Long id, String titulo, String conteudo) { this.id = id; this.titulo = titulo; this.conteudo = conteudo; } //
```

```
----- public void receberComentario(Comentario comentario) { this.comentarios.add(comentario); }
public List<Comentario> filtrarComentariosAprovados() { return comentarios.stream()
.filter(Comentario::isAprovado) .toList(); } public List<Comentario> filtrarComentariosPendentes() { return
comentarios.stream() .filter(c -> !c.isAprovado()) .toList(); } public void removerComentario(Comentario
comentario) { this.comentarios.remove(comentario); } public void receberAvaliacaoArtigo(double nota) {
this.quantidadeAvaliacoesArtigo++; this.somaAvaliacoesArtigo += nota; this.avaliacao =
this.somaAvaliacoesArtigo / this.quantidadeAvaliacoesArtigo; autor.avaliarReputacaoAutor(nota); } public
void incrementarVisualizacao(){ this.visualizacoes++; } //
```

```
----- @Override public String toString() { return String.format("Artigo {titulo= '%s', conteudo= '%s',
dataPublicacao= %s, autor= %s, comentarios= %d}", titulo, conteudo, dataPublicacao, autor.getNome(),
comentarios.size() ); } public Long getId() { return id; } public void setId(Long id) { this.id = id; } public String
getTitulo() { return titulo; } public void setTitulo(String titulo) { this.titulo = titulo; } public String
getConteudo() { return conteudo; } public void setConteudo(String conteudo) { this.conteudo = conteudo; }
public LocalDateTime getDataPublicacao() { return dataPublicacao; } public void
setDataPublicacao(LocalDateTime dataPublicacao) { this.dataPublicacao = dataPublicacao; } public boolean
isPublicado() { return publicado; } public void setPublicado(boolean publicado) { this.publicado = publicado;
} public int getVisualizacoes() { return visualizacoes; } public void setVisualizacoes(int visualizacoes) {
this.visualizacoes = visualizacoes; } public Autor getAutor() { return autor; } public void setAutor(Autor autor)
{ this.autor = autor; } public List<Comentario> getComentarios() { return comentarios; } public void
setComentarios(List<Comentario> comentarios) { this.comentarios = comentarios; } } package
br.edu.infnet.herick_bispo_blog_API.domain.model; import java.time.LocalDateTime; public class
Comentario implements Identificavel { private Long id; private String texto; private LocalDateTime
dataHora; private boolean aprovado; private Usuario autorComentario; public Comentario(){ public
Comentario(Long id, String texto) { this.id = id; this.texto = texto; } @Override public String toString() { return
```

```
String.format("Comentário {texto= '%s', autorComentario= %s, dataHora= %s}", texto,
autorComentario.getNome(), dataHora ); } public Long getId() { return id; } public void setId(Long id) { this.id
= id; } public String getTexto() { return texto; } public void setTexto(String texto) { this.texto = texto; } public
LocalDateTime getDataHora() { return dataHora; } public void setDataHora(LocalDateTime dataHora) {
this.dataHora = dataHora; } public boolean isAprovado() { return aprovado; } public void
setAprovado(boolean aprovado) { this.aprovado = aprovado; } public Usuario getAutorComentario() {
return autorComentario; } public void setAutorComentario(Usuario autorComentario) {
this.autorComentario = autorComentario; } } package br.edu.infnet.herick_bispo_blog_API.domain.model;
import java.time.LocalDateTime; import java.util.ArrayList; import java.util.List; public class Autor extends
Usuario{ private double reputacao = 5.0; private int quantidadeAvaliacoesEmArtigos = 0; private double
somaAvaliacoesEmArtigos = 0.0; private List<Artigo> artigos = new ArrayList<>(); public Autor(){ public
Autor(Long id,String nome, String email) { super(id, nome, email, LocalDateTime.now()); } //
```

```
----- public void publicarArtigo(Artigo artigo){ if(artigo == null){ throw new
IllegalArgumentException("O artigo não pode ser nulo"); } artigo.setDataPublicacao(LocalDateTime.now());
artigos.add(artigo); artigo.setAutor(this); artigo.setPublicado(true); } public void excluirArtigo(Artigo artigo,
boolean excluir){ if (excluir) { artigos.remove(artigo); } } public void avaliarReputacaoAutor(double
avaliacao){ quantidadeAvaliacoesEmArtigos ++; this.somaAvaliacoesEmArtigos += avaliacao; this.reputacao
= this.somaAvaliacoesEmArtigos / this.quantidadeAvaliacoesEmArtigos; } public void
moderarComentario(Artigo artigo, Comentario comentario, boolean aprovado) { // O autor só pode
moderar os próprios comentários if (!this.equals(artigo.getAutor())) { throw new
IllegalArgumentException("O autor só pode moderar comentários dos próprios artigos."); } if (aprovado) {
this.aprovar(artigo, comentario); } else { this.rejeitar(artigo, comentario); } } private void aprovar(Artigo
artigo, Comentario comentario) { comentario.setAprovado(true); } private void rejeitar(Artigo artigo,
Comentario comentario) { artigo.removeComentario(comentario); } //
```

```
----- @Override public String toString() { return String.format("Autor {%s, reputacao= '%s', artigos= '%d'",
super.toString(), reputacao, artigos.size() ); } public double getReputacao() { return reputacao; } public void
setReputacao(double reputacao) { this.reputacao = reputacao; } public List<Artigo> getArtigos() { return
artigos; } public void setArtigos(List<Artigo> artigos) { this.artigos = artigos; } } package
br.edu.infnet.herick_bispo_blog_API.domain.model; import java.time.LocalDateTime; public class Leitor
extends Usuario { private boolean inscritoNewsletter; public Leitor(){ public Leitor(Long id, String nome,
String email) { super(id, nome, email, LocalDateTime.now()); } //
```

```
----- public void comentar(Long id, Artigo artigo, String texto) { Comentario novoComentario = new
Comentario(id, texto); novoComentario.setAutorComentario(this);
novoComentario.setDataHora(LocalDateTime.now()); artigo.receberComentario(novoComentario); } public
void avaliar(Artigo artigo, double nota) { //A nota deverá ser entre 1 e 5 if(nota < 1 || nota > 5){ throw new
IllegalArgumentException("A nota deve ser entre 1 e 5"); } artigo.receberAvaliacaoArtigo(nota); } public void
assinarNewsLetter(Boolean inscrever){ if(inscrever){ this.inscritoNewsletter = true; }else {
this.inscritoNewsletter = false; } } //
```

```
----- @Override public String toString() { return String.format("Leitor {%s, inscritoNewsletter= '%s'}",
super.toString(), inscritoNewsletter ? "sim": "não" ); } public boolean isInscritoNewsletter() { return
inscritoNewsletter; } public void setInscritoNewsletter(boolean inscritoNewsletter) { this.inscritoNewsletter
= inscritoNewsletter; } } package br.edu.infnet.herick_bispo_blog_API.domain.model; import
java.time.LocalDateTime; public abstract class Usuario implements Identificavel { private Long id; private
String nome; private String email; private LocalDateTime dataCadastro; public Usuario(){ public
Usuario(Long id, String nome, String email, LocalDateTime dataCadastro) { this.id = id; this.nome = nome;
this.email = email; this.dataCadastro = dataCadastro; } @Override public String toString() { return
String.format("id= %d, nome= %s, email= %s, dataCadastro= %s", id, nome, email, dataCadastro ); } public
Long getId() { return id; } public void setId(Long id) { this.id = id; } public String getNome() { return nome; }
```

```
public void setNome(String nome) { this.nome = nome; } public String getEmail() { return email; } public void setEmail(String email) { this.email = email; } public LocalDateTime getDataCadastro() { return dataCadastro; } public void setDataCadastro(LocalDateTime dataCadastro) { this.dataCadastro = dataCadastro; } } package br.edu.infnet.herick_bispo_blog_API.domain.model; public interface Identificavel { Long getId(); }
```

Reavalie o código com o que já foi implementado, levando em consideração que: -Cálculo da Reputação do Autor: eu farei posteriormente na etapa 2 usando streams -Responsabilidade por Identificadores e Comparação de Objetos : entrará posteriormente na etapa 2

Response: Avaliação das Implementações Atuais

- **Unificação e Filtragem de Comentários:** A adoção de uma lista única e a filtragem utilizando **Streams** foram aplicadas corretamente. A lógica atual depende do valor padrão do tipo primitivo boolean no Java (que é false) para definir que novos comentários nascem pendentes. Isso funciona perfeitamente, embora em projetos maiores seja recomendado definir o estado explicitamente no construtor para maior clareza.
 - **Lógica de Rejeição de Comentários:** O seu método de rejeição remove o comentário da lista. Isso representa um **Hard Delete** (exclusão física). Em cenários reais, é comum aplicar o **Soft Delete** (exclusão lógica), mantendo o objeto na lista, mas alterando um status (por exemplo, criando um Enum com APROVADO, PENDENTE, REJEITADO) para fins de auditoria e histórico do blog. Para o escopo atual, a sua abordagem de remoção atende ao requisito lógico.
 - **Limites de Avaliação:** A correção da lógica de fronteira para as notas foi feita com sucesso. Agora a regra barra exclusivamente valores que estão de fato fora do intervalo desejado.
 - **Redundância de Código (Newsletter):** O método de assinar a newsletter possui um bloco condicional if/else redundante. Como o argumento recebido já é um valor booleano, você pode simplesmente atribuir o parâmetro recebido diretamente ao atributo da classe, resolvendo a lógica em uma única linha.
 - **Acoplamento de Entidades:** O método de receber avaliação do artigo chama diretamente o método do autor para atualizar sua reputação. Isso cria um **alto nível de acoplamento** (uma entidade coordenando o comportamento da outra). Conforme você evoluir para a arquitetura final da API, essa orquestração de atualização dupla será responsabilidade ideal da camada de **Service**, deixando as entidades focadas apenas em seus próprios estados.
-